Obliczenia ewolucyjne

Projekt 1

Implementacja klasycznego algorytmu genetycznego

Wykonali:

Łukasz Ryczko
Maciej Malina

Październik 2026

Spis treści

1 Cel projektu **2**

2 Wykorzystane technologie **2**

3 Wymagania środowiska i uruchomienie **2**

4 Architektura aplikacji **3**

5 Implementacja algorytmu **3**
5.1 Reprezentacja chromosomu . . . 3
5.2 Przystosowanie . . . 4
5.3 Selekcja . . . 4
5.4 Krzyżowanie . . . 4
5.5 Mutacja, inwersja i strategia elitarna . . . 4
5.6 Przebieg jednej epoki . . . 4

6 Interfejs graficzny **5**
6.1 Zapis wyników . . . 8

7 Wybrane funkcje testowe **8**
7.1 Ackley . . . 9
7.2 Schwefel . . . 9
7.3 CEC2014 F4: Shifted and Rotated Rosenbrock's Function . . . 9
7.4 CEC2014 F8: Shifted Rastrigin's Function . . . 10
7.5 Wartości optimów . . . 10

8 Eksperymenty **11**
8.1 Metodyka . . . 11
8.2 Ackley, 10 zmiennych . . . 11
8.3 Schwefel, 10 zmiennych . . . 13
8.4 CEC2014 F4, 10 zmiennych . . . 14
8.5 CEC2014 F8, 10 zmiennych . . . 15
8.6 Porównanie konfiguracji . . . 16
8.7 Porównanie czasu obliczeń . . . 16
8.8 Wpływ liczby zmiennych . . . 16

9 Wnioski **17**

10 Podsumowanie **17**

1 Cel projektu

Celem projektu było zaimplementowanie klasycznego algorytmu genetycznego z binarną reprezentacją chromosomu, który pozwala minimalizować oraz maksymalizować funkcje dowolnej liczby zmiennych. Aplikacja ma interfejs graficzny, w którym można skonfigurować wszystkie parametry algorytmu, obserwować przebieg obliczeń na wykresach i odczytać czas wykonania. Wyniki każdej epoki są zapisywane w bazie danych SQLite.

Zaimplementowane elementy:

- binarna reprezentacja chromosomu z konfigurowalną dokładnością,
- konfiguracja wielkości populacji, liczby epok i liczby zmiennych,
- selekcja najlepszych, najgorszych, kołem ruletki i turniejowa (z parametrami),
- krzyżowanie jednopunktowe, dwupunktowe, jednorodne i ziarniste (z prawdopodobieństwem),
- mutacja brzegowa, jednopunktowa i dwupunktowa (z prawdopodobieństwem),
- operator inwersji (z prawdopodobieństwem),
- strategia elitarna (ułamek populacji albo liczba osobników),
- wykresy wartości funkcji oraz średniej i odchylenia standardowego w kolejnych epokach,
- zapis wyników każdej epoki do bazy danych SQLite.

2 Wykorzystane technologie

Technologia	Zastosowanie
Python 3.12	język implementacji całego projektu
Tkinter / ttk (Tk 9.1)	interfejs graficzny
NumPy	operacje na populacji (macierze bitów), dekodowanie chromosomów, statystyki
Matplotlib	wykresy w aplikacji (osadzone w Tkinter) i w sprawozdaniu
benchmark_functions 1.1.4	gotowe implementacje prostych funkcji testowych (L. Baronti)
opfunu 1.0.1	gotowe implementacje funkcji CEC 2014
SciPy	zależność biblioteki benchmark_functions
SQLite (moduł `sqlite3`)	zapis wyników kolejnych epok i eksperymentów
Git	kontrola wersji
LaTeX (Tectonic)	sprawozdanie

Tabela 1: Technologie użyte w projekcie.

3 Wymagania środowiska i uruchomienie

- system macOS, Linux lub Windows,
- Python w wersji 3.10 lub nowszej z obsługą Tkinter,
- pakiety z pliku `app/requirements.txt`: numpy, scipy, matplotlib, benchmark_functions, opfunu oraz setuptools w wersji niższej niż 81 (opfunu korzysta z modułu `pkg_resources`, usuniętego w nowszych wersjach setuptools).

Na macOS z Homebrew Tkinter instaluje się poleceniem `brew install python-tk@3.12`. Systemowy Python z macOS ma przestarzały Tk 8.5, który na nowych wersjach systemu źle się renderuje.

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r app/requirements.txt
python app/main.py
```

Eksperymenty do sprawozdania (wszystkie tabele i wykresy z rozdziału 8) można odtworzyć poleceniem `python -m experiments.run_experiments` uruchomionym w katalogu `app/`.

4 Architektura aplikacji

Aplikacja została zaprojektowana obiektowo. Każdy operator genetyczny jest osobną klasą implementującą wspólny interfejs (wzorzec Strategia), dzięki czemu algorytm nie zależy od konkretnych metod selekcji, krzyżowania czy mutacji. Funkcje z dwóch różnych bibliotek są opakowane adapterami o jednym interfejsie, a obiekty są składane przez fabrykę na podstawie konfiguracji.

Moduł	Zawartość
`ga/config.py`	`GAConfig`: niezmienna konfiguracja z walidacją parametrów
`ga/encoding.py`	`BinaryEncoder`: liczba bitów, losowanie populacji, dekodowanie
`ga/selection.py`	`SelectionMethod` oraz `BestSelection`, `WorstSelection`, `RouletteSelection`, `TournamentSelection`
`ga/crossover.py`	`CrossoverMethod` oraz krzyżowanie jednopunktowe, dwupunktowe, jednorodne, ziarniste
`ga/mutation.py`	`MutationMethod` oraz mutacja brzegowa, jednopunktowa, dwupunktowa
`ga/inversion.py`	`InversionOperator`
`ga/elitism.py`	`EliteStrategy`
`ga/algorithm.py`	`GeneticAlgorithm`: pętla epok, statystyki `EpochStats`, wynik `RunResult`
`ga/factory.py`	rejestry operatorów i fabryka `build_algorithm(config)`
`functions/`	`ObjectiveFunction` oraz adaptery `BenchmarkFunctionAdapter` i `CecFunctionAdapter`, rejestr funkcji
`storage/`	`SqliteResultRepository`: zapis wyników do bazy SQLite (wzorzec Repozytorium)
`gui/`	okno główne, formularz parametrów, panel wykresów
`experiments/`	skrypty eksperymentów i generowania tabel oraz wykresów do sprawozdania

Tabela 2: Podział kodu na moduły (katalog `app/`).

Obliczenia działają w osobnym wątku. Po każdej epoce algorytm przekazuje obiekt `EpochStats` przez kolejkę do wątku interfejsu, który odświeża karty z wynikami i wykresy. Dzięki temu okno nie zawiesza się w trakcie obliczeń, a obliczenia można przerwać przyciskiem Stop.

5 Implementacja algorytmu

5.1 Reprezentacja chromosomu

Dla przedziału $[a, b]$ i dokładności ε liczba bitów na jedną zmienną to

$$m = \left\lceil \log_2\left(\frac{b-a}{\varepsilon} + 1\right) \right\rceil,$$

a chromosom dla n zmiennych ma $n \cdot m$ bitów. Dekodowanie i-tej zmiennej:

$$x_i = a + \mathrm{dec}(g_i) \cdot \frac{b-a}{2^m - 1},$$

gdzie $\mathrm{dec}(g_i)$ to wartość dziesiętna i-tego fragmentu chromosomu. Cała populacja jest przechowywana jako macierz bitów NumPy, a dekodowanie odbywa się jednym mnożeniem macierzy. Przykładowo dla przedziału $[-100, 100]$ i dokładności 10^{-4} każda zmienna ma 21 bitów.

5.2 Przystosowanie

Wszystkie operatory porównują osobniki według przystosowania w konwencji „im więcej, tym lepiej". Przy maksymalizacji przystosowaniem jest $f(x)$, przy minimalizacji $-f(x)$. Ten sam kod obsługuje więc oba rodzaje optymalizacji.

5.3 Selekcja

- **Selekcja najlepszych**: do puli rodzicielskiej trafia zadany ułamek najlepszych osobników.
- **Selekcja najgorszych**: do puli trafia zadany ułamek najsłabszych osobników.
- **Koło ruletki**: prawdopodobieństwo wyboru jest proporcjonalne do przystosowania przesuniętego tak, by najgorszy osobnik miał wagę zero. Działa dzięki temu dla minimalizacji i dla wartości ujemnych.
- **Selekcja turniejowa**: losujemy k osobników (rozmiar turnieju), do puli trafia najlepszy z nich.

5.4 Krzyżowanie

Para rodziców losowanych z puli jest krzyżowana z zadanym prawdopodobieństwem, w przeciwnym razie potomkowie są kopiami rodziców.

- **Jednopunktowe**: wymiana końcówek chromosomów od losowego punktu.
- **Dwupunktowe**: wymiana fragmentu pomiędzy dwoma losowymi punktami.
- **Jednorodne**: każdy bit niezależnie z prawdopodobieństwem 0.5 jest zamieniany między rodzicami.
- **Ziarniste**: chromosom dzielony jest na ziarna (bloki) o konfigurowalnej długości, każde ziarno potomek dziedziczy w całości od losowo wybranego rodzica.

5.5 Mutacja, inwersja i strategia elitarna

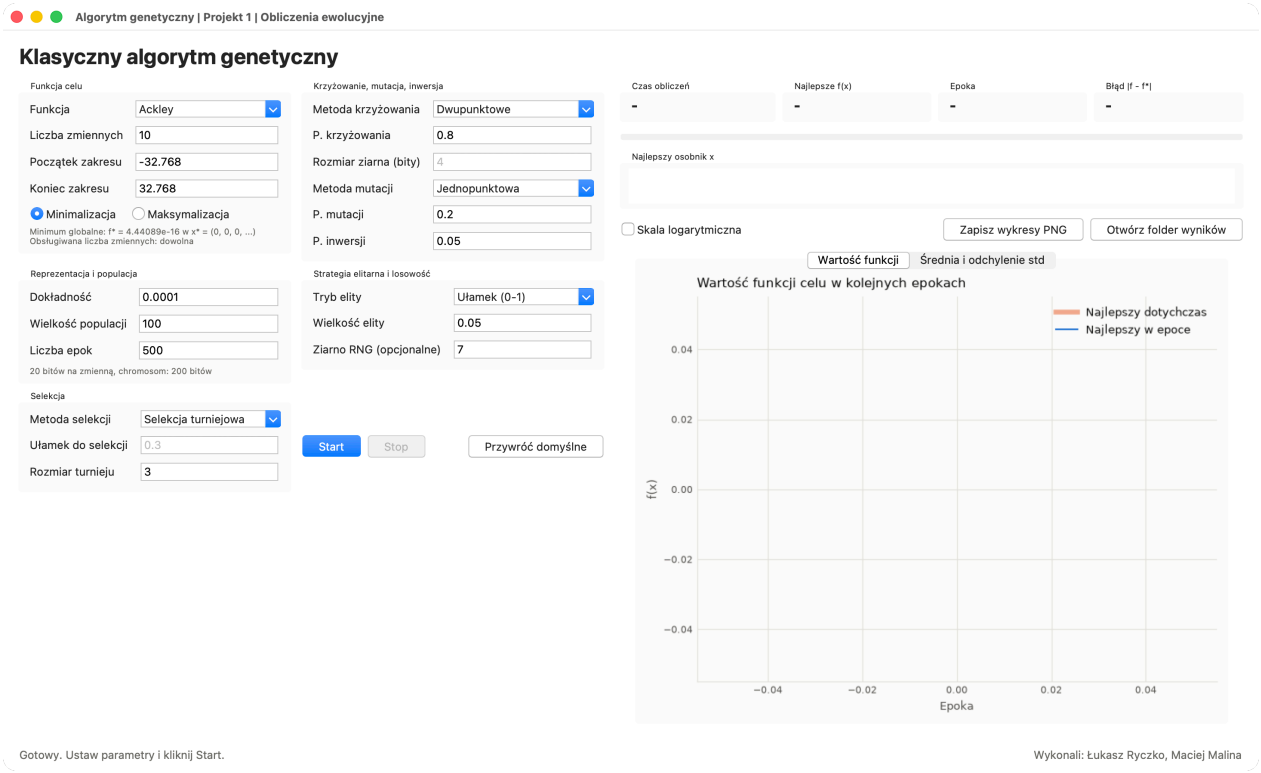
- **Mutacja brzegowa**: negacja pierwszego albo ostatniego bitu chromosomu.
- **Mutacja jednopunktowa**: negacja jednego losowego bitu.
- **Mutacja dwupunktowa**: negacja dwóch różnych losowych bitów.
- **Inwersja**: z zadanym prawdopodobieństwem odwracana jest kolejność bitów pomiędzy dwoma losowymi punktami.
- **Strategia elitarna**: najlepsze osobniki (ułamek populacji albo podana liczba) przechodzą bez zmian do następnej populacji.

5.6 Przebieg jednej epoki

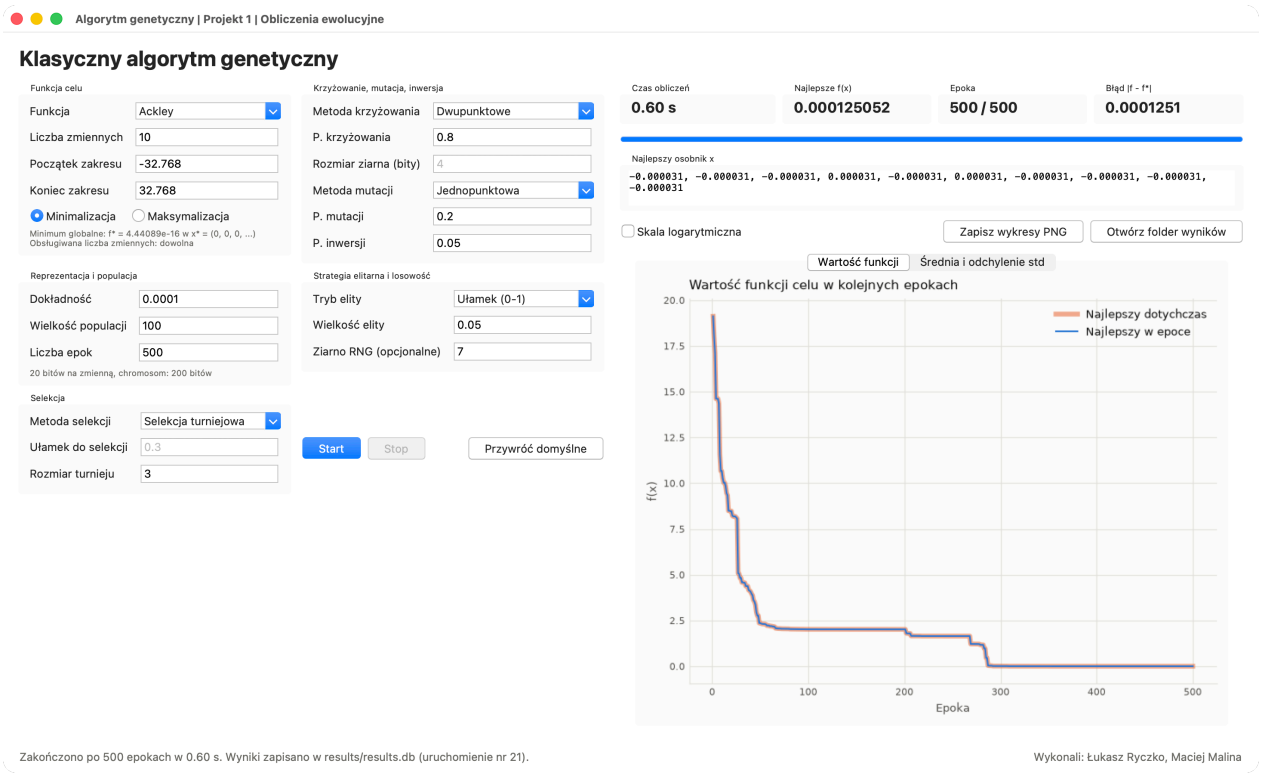
1. Dekodowanie populacji i obliczenie wartości funkcji celu.
2. Zapisanie statystyk: najlepsza wartość w epoce, najlepsza dotychczas, średnia, odchylenie standardowe.
3. Wybór elity.
4. Selekcja puli rodzicielskiej.
5. Tworzenie potomków (krzyżowanie, mutacja, inwersja), aż populacja wraz z elitą osiągnie zadaną wielkość.

6 Interfejs graficzny

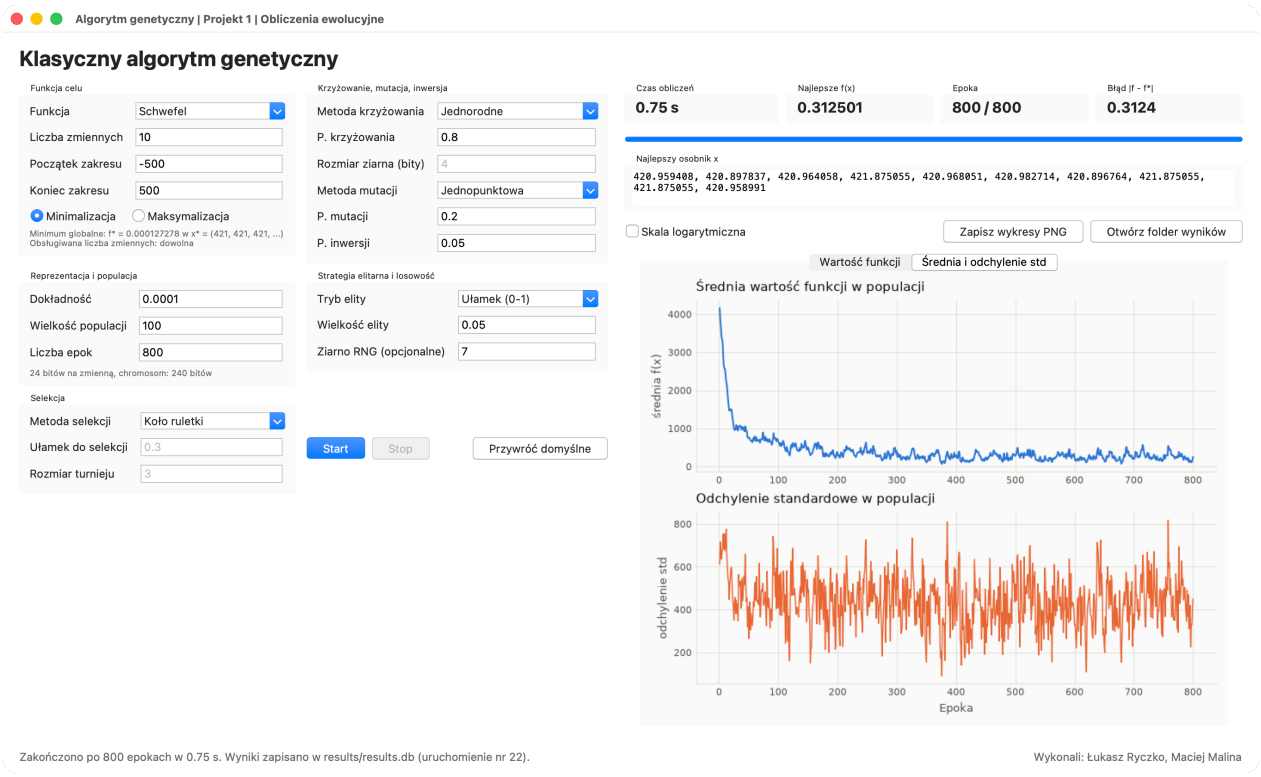
Okno aplikacji składa się z formularza parametrów (po lewej) i panelu wyników (po prawej). W panelu wyników widać czas obliczeń, najlepszą znalezioną wartość, numer epoki, błąd względem znanego optimum, najlepszego osobnika oraz dwa wykresy aktualizowane na żywo. Pod polem wyboru funkcji wyświetlane jest jej optimum oraz dozwolona liczba zmiennych, a pod dokładnością liczba bitów na zmienną i długość chromosomu. Pola nieużywane przez wybraną metodę (np. rozmiar turnieju przy kole ruletki) są automatycznie wyłączane. Niepoprawne dane (np. ujemna dokładność albo koniec zakresu mniejszy od początku) powodują wyświetlenie komunikatu z opisem błędu.



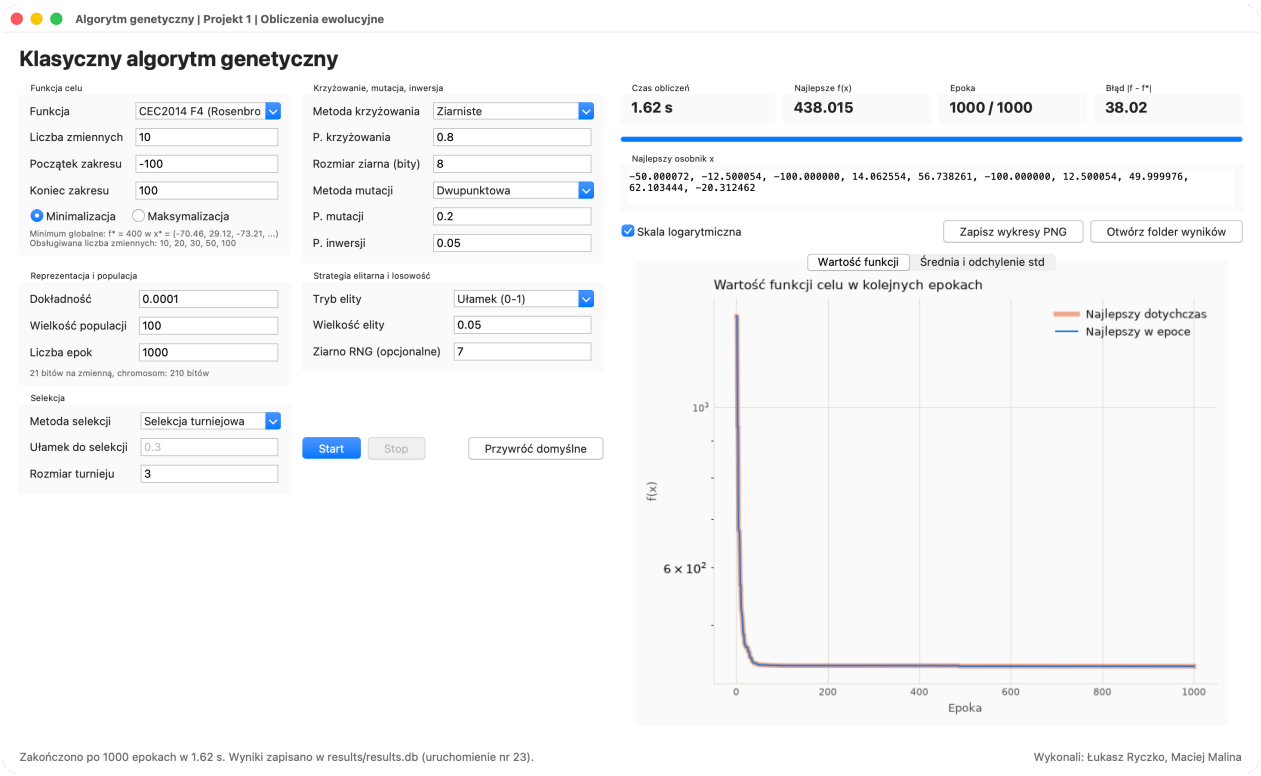
Rysunek 1: Aplikacja po uruchomieniu, przed rozpoczęciem obliczeń.



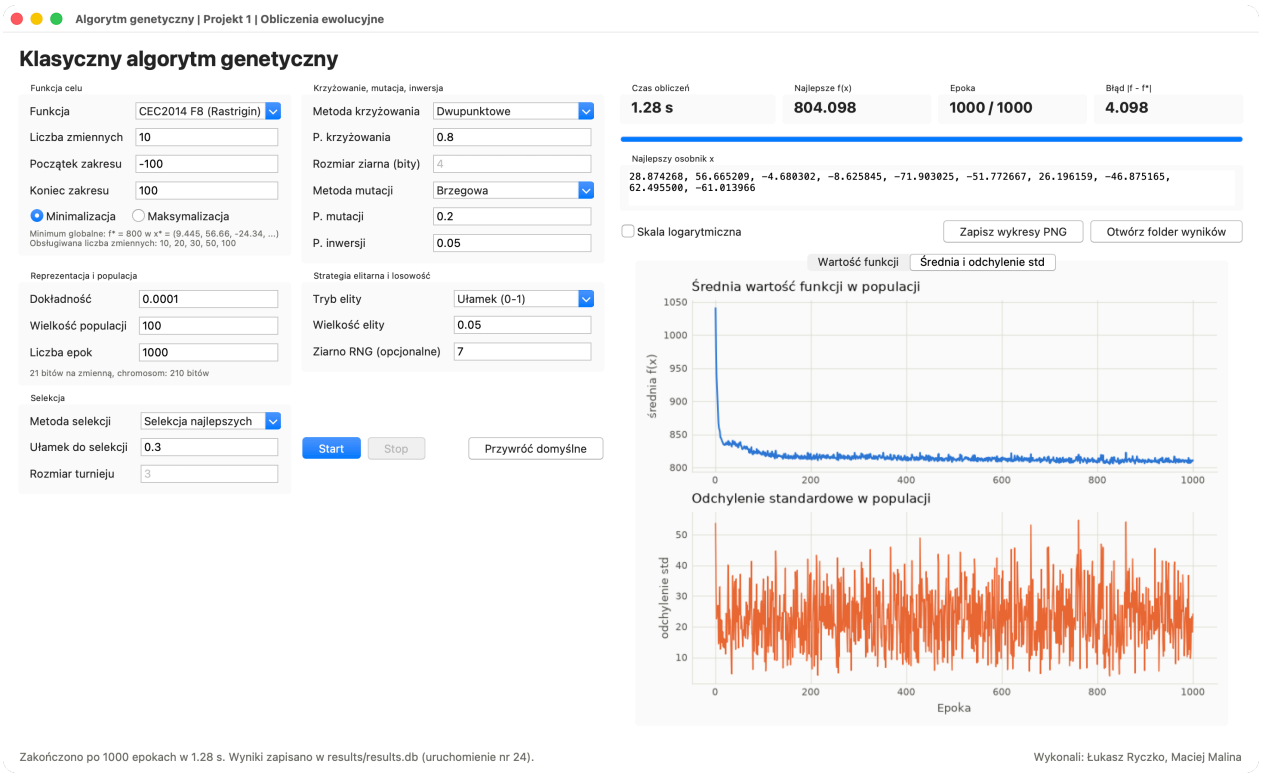
Rysunek 2: Minimalizacja funkcji Ackley dla 10 zmiennych (selekcja turniejowa, krzyżowanie dwupunktowe).



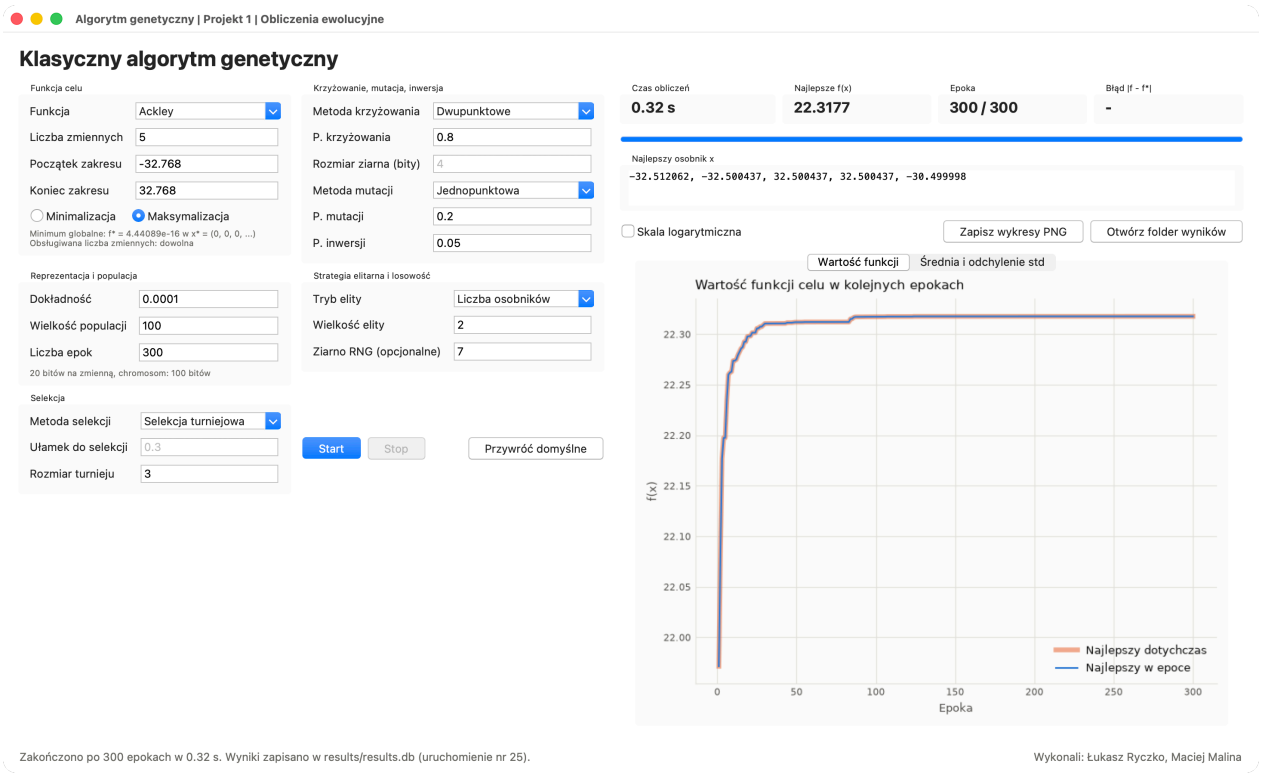
Rysunek 3: Funkcja Schwefel, 10 zmiennych: koło ruletki i krzyżowanie jednorodne. Zakładka ze średnią i odchyleniem standardowym.



Rysunek 4: CEC2014 F4, 10 zmiennych: krzyżowanie ziarniste, mutacja dwupunktowa, skala logarytmiczna.



Rysunek 5: CEC2014 F8, 10 zmiennych: selekcja najlepszych, mutacja brzegowa.



Rysunek 6: Maksymalizacja funkcji Ackley dla 5 zmiennych, elita podana jako liczba osobników (2). Algorytm znajduje punkty w pobliżu krawędzi przedziału.

6.1 Zapis wyników

Każde uruchomienie jest zapisywane w bazie SQLite `app/results/results.db`, która ma dwie tabele:

- `runs`: jeden wiersz podsumowania na uruchomienie (data, funkcja, liczba zmiennych, najlepszy wynik, czas, liczba epok, czy przerwano, najlepszy punkt i pełna konfiguracja w formacie JSON),
- `epochs`: wiersz dla każdej epoki z kolumnami `run_id`, `epoch`, `best`, `mean`, `std`, `best_so_far`, `best_x` (klucz obcy `run_id` wskazuje wiersz w tabeli `runs`).

Przykładowe zapytanie zwracające przebieg ostatniego uruchomienia:

```
SELECT epoch, best, mean, std FROM epochs
WHERE run_id = (SELECT MAX(id) FROM runs) ORDER BY epoch;
```

Przycisk „Zapisz wykresy PNG” zapisuje oba wykresy do `app/results/plots/`.

7 Wybrane funkcje testowe

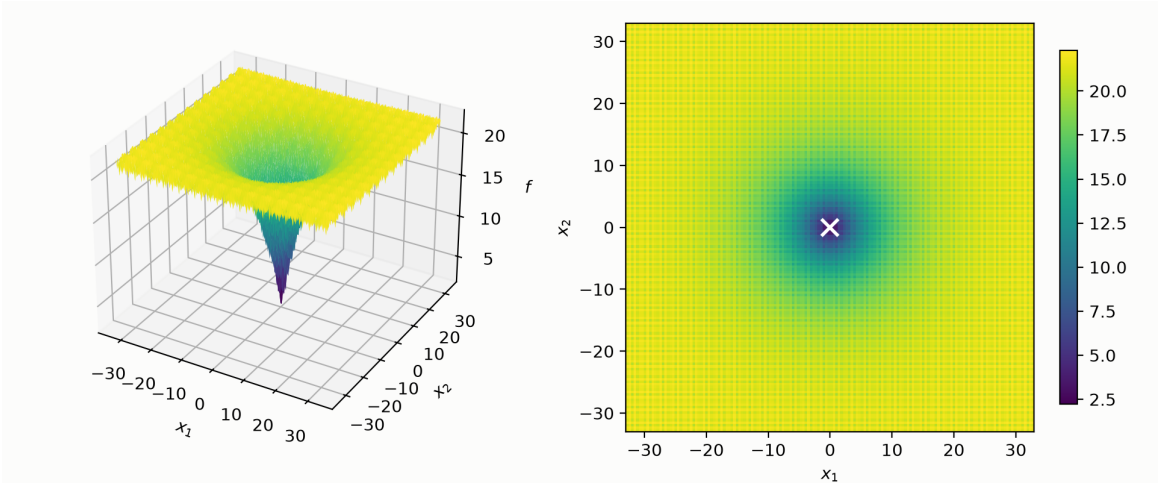
Wybraliśmy dwie proste funkcje z biblioteki benchmark_functions (Ackley i Schwefel) oraz dwie funkcje z zestawu CEC 2014 z biblioteki opfunu (F4 i F8). Wszystkie pozwalają ustawić różną liczbę zmiennych (funkcje CEC 2014 tylko 10, 20, 30, 50 lub 100).

Początkowo rozważaliśmy funkcję CEC2014 F10 (Shifted Schwefel). Okazało się jednak, że jej implementacja w opfunu 1.0.1 ma błąd znaku w obsłudze punktów spoza przedziału $[-500, 500]$: algorytm znajdował wartości około -3000, czyli mniejsze od optimum równego 1000. Z tego powodu wybraliśmy funkcję F8.

7.1 Ackley

$$f(x) = -20\exp\left(-0.2\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}\right) - \exp\left(\frac{1}{n}\sum_{i=1}^{n}\cos(2\pi x_i)\right) + 20 + e, \qquad x_i \in [-32.768,\ 32.768].$$

Minimum globalne $f(x^*) = 0$ w punkcie $x^* = (0, \ldots, 0)$. Funkcja ma bardzo wiele minimów lokalnych, ale wyraźne globalne „lejkowate" minimum.

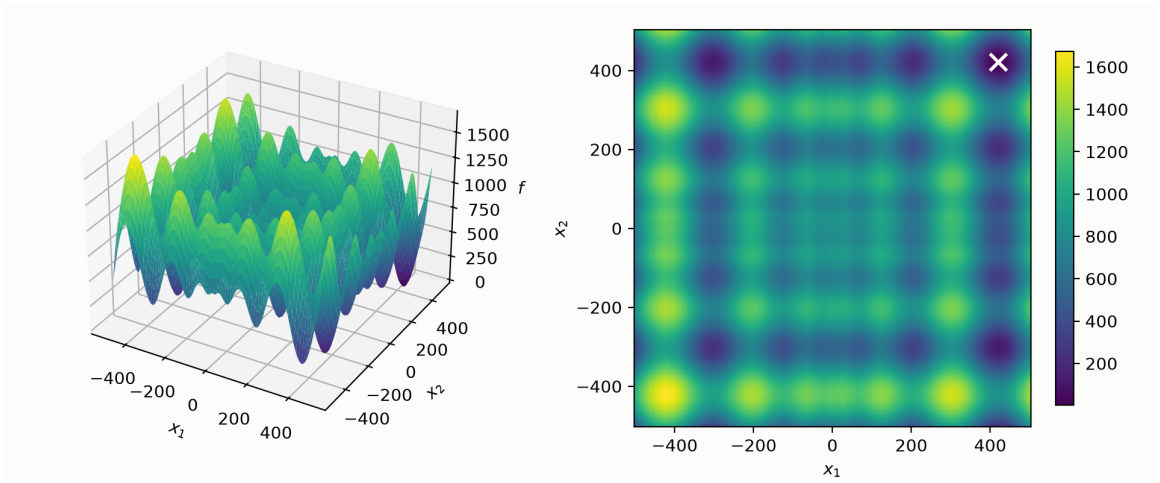


Rysunek 7: Funkcja Ackley dla 2 zmiennych: wykres 3D i mapa ciepła (krzyżyk oznacza optimum).

7.2 Schwefel

$$f(x) = 418.9829\,n - \sum_{i=1}^{n} x_i \sin\left(\sqrt{|x_i|}\right), \qquad x_i \in [-500,\ 500].$$

Minimum globalne $f(x^*) \approx 0$ w punkcie $x^* = (420.9687, \ldots, 420.9687)$. Najlepsze minimum lokalne leży daleko od globalnego, przy przeciwnym końcu przedziału, co utrudnia optymalizację.

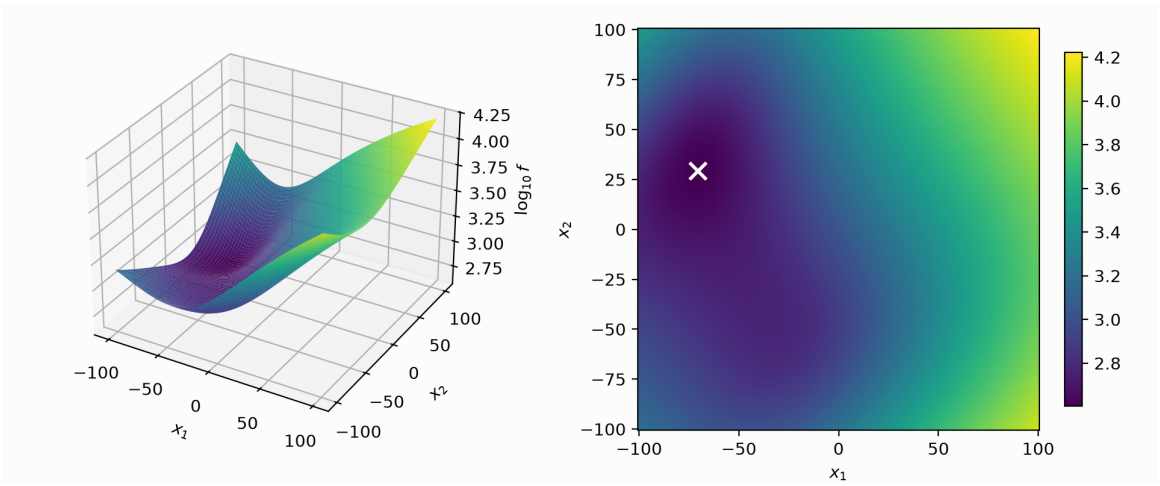


Rysunek 8: Funkcja Schwefel dla 2 zmiennych.

7.3 CEC2014 F4: Shifted and Rotated Rosenbrock's Function

$$F_4(x) = f_4\left(\mathbf{M}_4\left(\frac{2.048\,(x - o_4)}{100}\right) + 1\right) + 400, \qquad f_4(z) = \sum_{i=1}^{D-1}\left(100\,(z_i^2 - z_{i+1})^2 + (z_i - 1)^2\right),$$

gdzie o_4 to wektor przesunięcia, a $\mathbf{M}_4$ macierz obrotu. Przedział $[-100, 100]^D$, minimum $F_4(x^*) = 400$ w punkcie $x^* = o_4$. Obrót powoduje, że zmienne są ze sobą powiązane, a dno „doliny" Rosenbrocka jest bardzo płaskie.

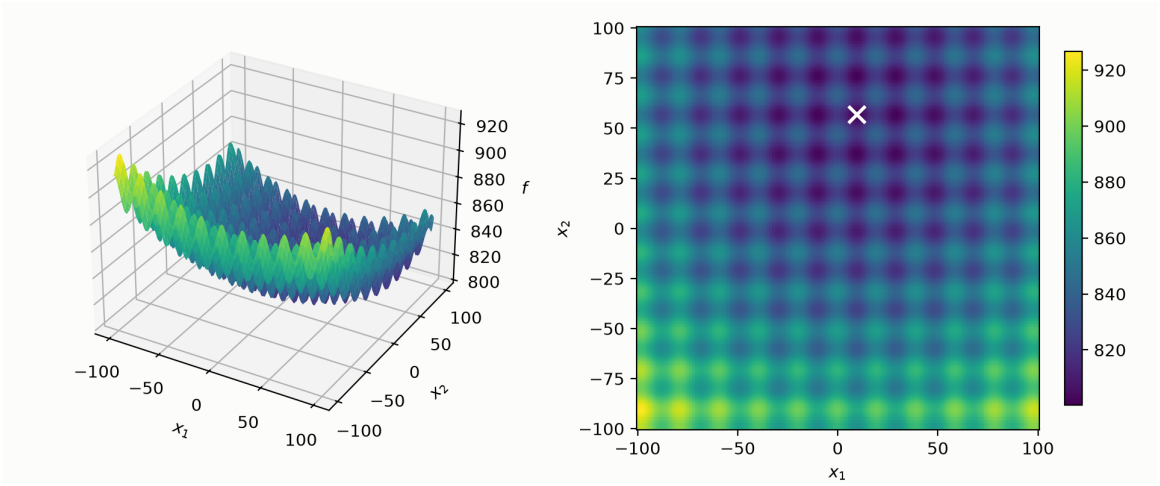


Rysunek 9: CEC2014 F4: przekrój funkcji 10 zmiennych przez optimum w płaszczyźnie (x_1, x_2), pozostałe zmienne ustalone na wartościach optymalnych. Oś pionowa w skali $\log_{10}$.

7.4 CEC2014 F8: Shifted Rastrigin's Function

$$F_8(x) = f_8\left(\frac{5.12\,(x - o_8)}{100}\right) + 800, \qquad f_8(z) = \sum_{i=1}^{D} \left(z_i^2 - 10\cos(2\pi z_i) + 10\right).$$

Przedział $[-100, 100]^D$, minimum $F_8(x^*) = 800$ w punkcie $x^* = o_8$. Funkcja jest separowalna, ale ma ogromną liczbę regularnie rozłożonych minimów lokalnych.



Rysunek 10: CEC2014 F8: przekrój funkcji 10 zmiennych przez optimum w płaszczyźnie (x_1, x_2).

7.5 Wartości optimów

Wartości optimum dla wszystkich badanych liczb zmiennych zostały odczytane z bibliotek. Dla Ackley biblioteka zwraca $4.4 \cdot 10^{-16}$, czyli zero z dokładnością do błędu zaokrągleń. Dla Schwefel stała 418.9829 jest zaokrąglona, dlatego minimum wynosi około $1.3 \cdot 10^{-5}\,n$ zamiast dokładnie 0. Dla funkcji CEC punkt optimum jest wektorem przesunięcia o; pokazujemy jego pierwsze trzy współrzędne (pełne wektory są w plikach danych biblioteki opfunu).

Funkcja	n	$f(x^*)$	x^*
Ackley	10	$4.441 \cdot 10^{-16}$	$(0.0000, 0.0000, 0.0000, \ldots)$
Ackley	20	$4.441 \cdot 10^{-16}$	$(0.0000, 0.0000, 0.0000, \ldots)$
Ackley	30	$4.441 \cdot 10^{-16}$	$(0.0000, 0.0000, 0.0000, \ldots)$
Schwefel	10	$1.273 \cdot 10^{-4}$	$(420.9687, 420.9687, 420.9687, \ldots)$
Schwefel	20	$2.546 \cdot 10^{-4}$	$(420.9687, 420.9687, 420.9687, \ldots)$
Schwefel	30	$3.818 \cdot 10^{-4}$	$(420.9687, 420.9687, 420.9687, \ldots)$
CEC2014 F4	10	400.0000	$(-70.4610, 29.1155, -73.2110, \ldots)$
CEC2014 F4	20	400.0000	$(-70.4610, 29.1155, -73.2110, \ldots)$
CEC2014 F4	30	400.0000	$(-70.4610, 29.1155, -73.2110, \ldots)$
CEC2014 F8	10	800.0000	$(9.4452, 56.6560, -24.3393, \ldots)$
CEC2014 F8	20	800.0000	$(9.4452, 56.6560, -24.3393, \ldots)$
CEC2014 F8	30	800.0000	$(9.4452, 56.6560, -24.3393, \ldots)$

Tabela 3: Optima globalne wybranych funkcji dla 10, 20 i 30 zmiennych.

8 Eksperymenty

8.1 Metodyka

Konfiguracja bazowa (dla wszystkich funkcji jednakowa): populacja 100, 1000 epok, dokładność 10^{-4}, selekcja turniejowa (rozmiar 3), krzyżowanie dwupunktowe ($p_k = 0.8$), mutacja jednopunktowa ($p_m = 0.2$), inwersja ($p_i = 0.05$), elita 5% populacji. Ułamek dla selekcji najlepszych i najgorszych wynosi 0.3, rozmiar ziarna w krzyżowaniu ziarnistym 8 bitów.

W każdym wariancie zmieniamy względem konfiguracji bazowej tylko jeden element (metodę selekcji, krzyżowania, mutacji albo wyłączamy elitę), dzięki czemu widać wpływ tego elementu na wynik. Każda konfiguracja została uruchomiona 10 razy z różnymi ziarnami generatora liczb losowych. W tabelach podajemy średni, najlepszy i najgorszy wynik z 10 uruchomień, odchylenie standardowe oraz średni czas jednego uruchomienia. Pogrubiono najlepszą średnią. Wszystkie 480 uruchomień wykonano kolejno w jednym procesie na tym samym komputerze (MacBook z procesorem Apple Silicon), dzięki czemu zmierzone czasy można bezpośrednio porównywać. Wyniki pojedynczych uruchomień (wynik i czas) są w bazie `report/data/experiments.db`.

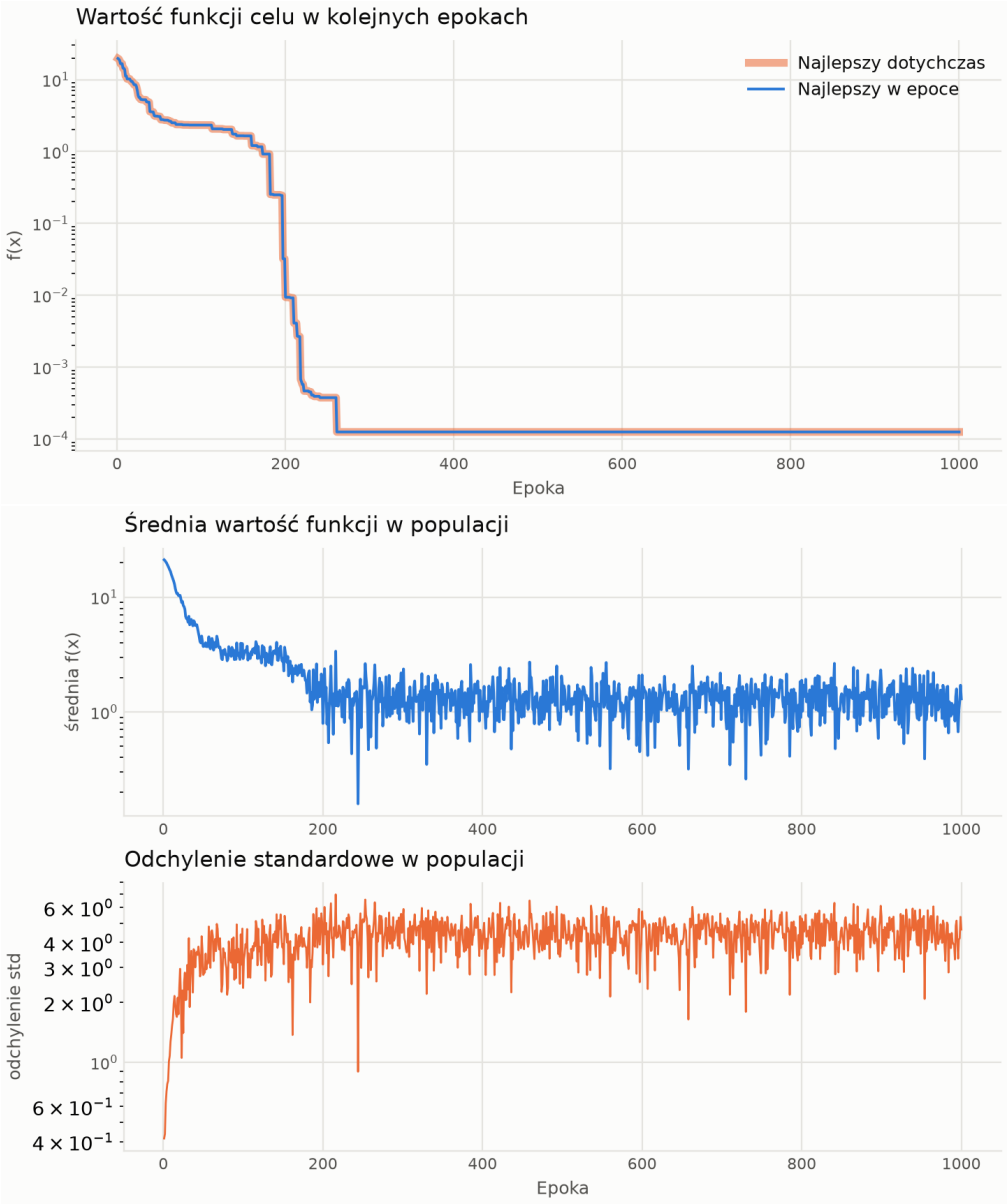
Wykresy wartości funkcji celu oraz średniej i odchylenia standardowego w kolejnych epokach zamieszczamy tylko dla najlepszego uruchomienia danej funkcji, czyli uruchomienia z najmniejszym wynikiem spośród wszystkich 100 uruchomień przy 10 zmiennych (10 konfiguracji po 10 powtórzeń).

8.2 Ackley, 10 zmiennych

Konfiguracja	Średnia	Najlepszy	Najgorszy	Odch. std	Czas [s]
Bazowa	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$5.037 \cdot 10^{-15}$	0.71
Selekcja najlepszych	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$2.864 \cdot 10^{-15}$	0.71
Selekcja najgorszych	19.4365	19.0220	20.0042	0.3479	0.72
Koło ruletki	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$4.437 \cdot 10^{-15}$	0.73
Krzyżowanie 1-punktowe	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$4.437 \cdot 10^{-15}$	0.60
Krzyżowanie jednorodne	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$3.709 \cdot 10^{-15}$	0.63
Krzyżowanie ziarniste	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$4.204 \cdot 10^{-15}$	0.60
Mutacja brzegowa	1.5422	0.2700	3.3273	0.7402	0.68
Mutacja 2-punktowa	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$4.619 \cdot 10^{-15}$	0.74
Bez elitaryzmu	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$4.780 \cdot 10^{-15}$	0.77

Tabela 4: Ackley, 10 zmiennych, 10 uruchomień każdej konfiguracji ($f^* = 0$).

Prawie wszystkie konfiguracje we wszystkich uruchomieniach osiągają ten sam wynik $1.251 \cdot 10^{-4}$. Jest to granica wynikająca z dokładności reprezentacji. Przy dokładności 10^{-4} siatka punktów na przedziale funkcji Ackley nie zawiera dokładnie zera, a najbliższy punkt to około $\pm 3.1 \cdot 10^{-5}$. Mniejszy błąd można uzyskać, zwiększając dokładność.

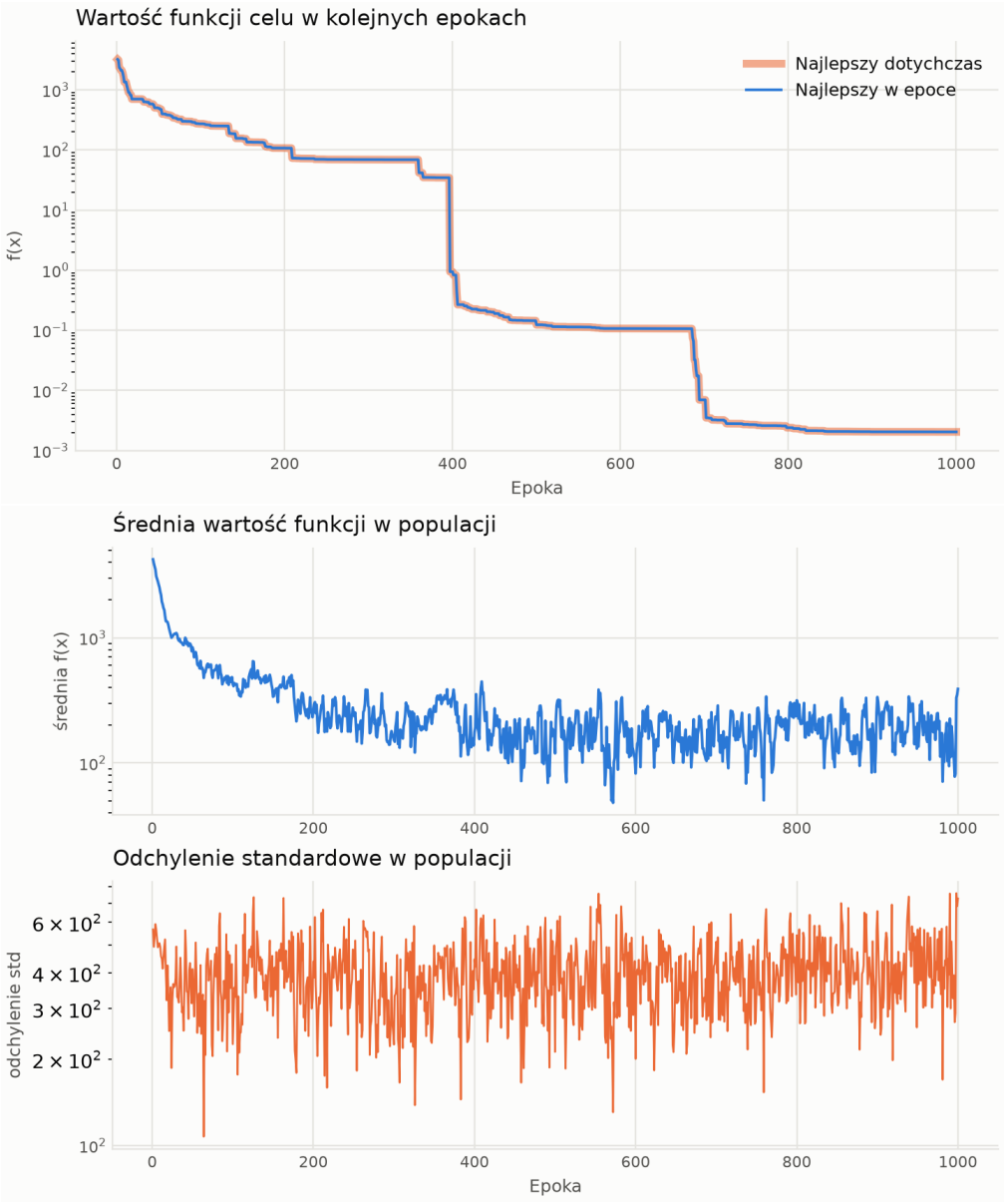


Rysunek 11: Ackley, najlepsze uruchomienie (konfiguracja „Bazowa", uruchomienie 5 z 10, wynik $1.251 \cdot 10^{-4}$, czas 0.70 s). Wartość funkcji, średnia i odchylenie standardowe w kolejnych epokach (skala logarytmiczna).

8.3 Schwefel, 10 zmiennych

Konfiguracja	Średnia	Najlepszy	Najgorszy	Odch. std	Czas [s]
Bazowa	113.3686	0.3131	347.8354	95.7280	0.67
Selekcja najlepszych	133.9766	0.4174	305.8667	92.0093	0.67
Selekcja najgorszych	2680.6541	2341.7394	2977.7151	175.5900	0.67
Koło ruletki	**12.1174**	0.0020	119.0607	35.6482	0.67
Krzyżowanie 1-punktowe	104.5221	0.1067	302.8461	84.5547	0.54
Krzyżowanie jednorodne	96.2235	0.1057	273.0456	97.8133	0.57
Krzyżowanie ziarniste	75.6598	0.3121	237.3960	69.1475	0.57
Mutacja brzegowa	62.4122	0.5961	189.6658	72.6109	0.66
Mutacja 2-punktowa	15.9069	0.1051	69.1973	22.3475	0.72
Bez elitaryzmu	143.7129	27.1954	434.7897	124.1654	0.67

Tabela 5: Schwefel, 10 zmiennych ($f^* \approx 1.3 \cdot 10^{-4}$).

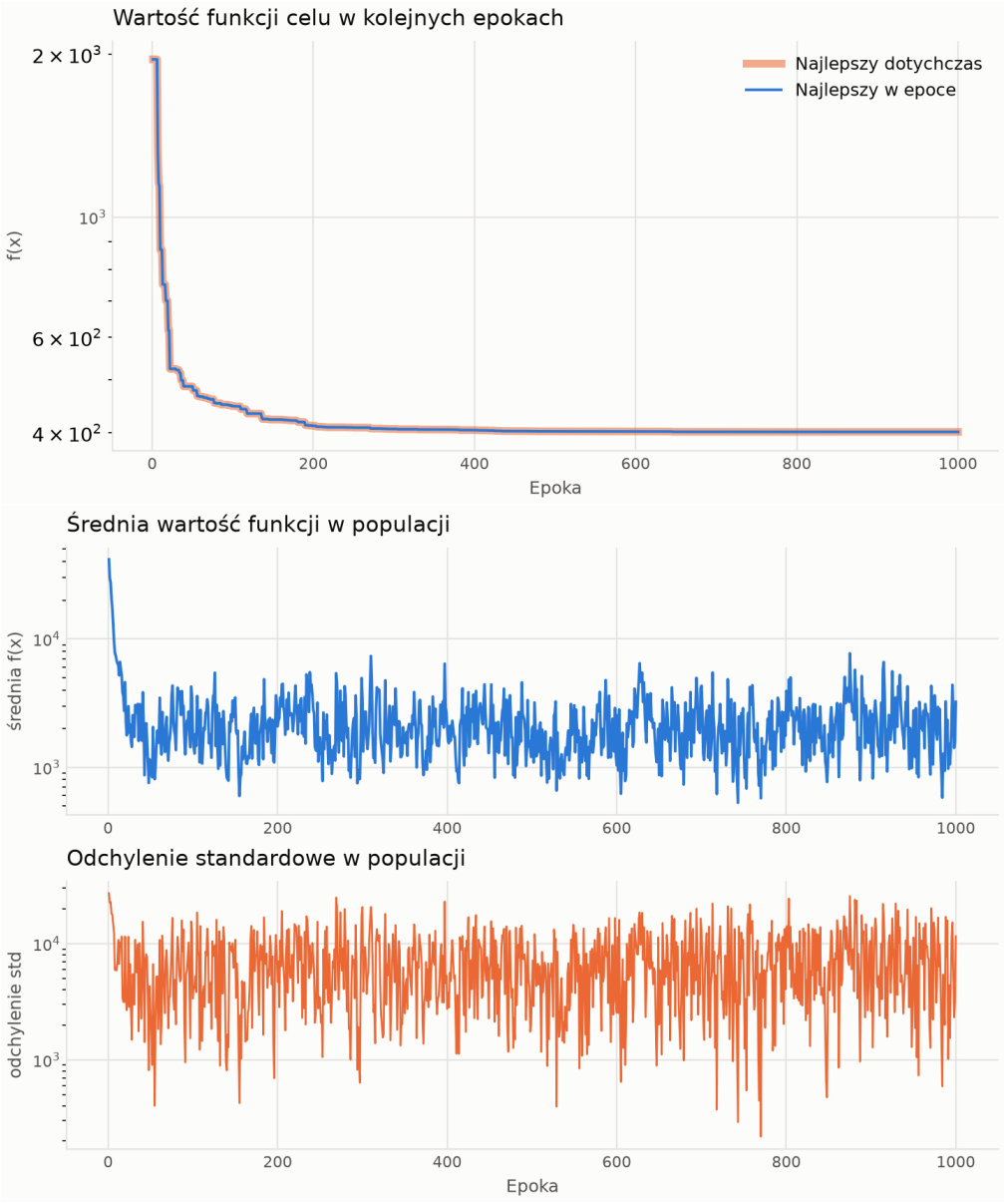


Rysunek 12: Schwefel, najlepsze uruchomienie (konfiguracja „Koło ruletki”, uruchomienie 10 z 10, wynik 0.0020, czas 0.67 s).

8.4 CEC2014 F4, 10 zmiennych

Konfiguracja	Średnia	Najlepszy	Najgorszy	Odch. std	Czas [s]
Bazowa	444.4991	419.1238	469.0697	15.5177	0.85
Selekcja najlepszych	440.9716	407.5054	477.9413	17.5938	0.86
Selekcja najgorszych	2531.5247	1526.2052	4122.9294	700.8951	0.87
Koło ruletki	**431.7961**	401.0450	470.8068	23.4115	0.87
Krzyżowanie 1-punktowe	446.5250	410.3174	480.7700	20.9320	0.73
Krzyżowanie jednorodne	441.4051	407.7439	463.0526	15.0571	0.77
Krzyżowanie ziarniste	437.6815	402.7363	470.2106	20.9870	0.76
Mutacja brzegowa	433.4924	411.6873	472.4615	17.6817	0.91
Mutacja 2-punktowa	436.1566	403.0928	477.2560	20.5021	0.91
Bez elitaryzmu	439.2992	406.2288	473.2319	21.2775	0.89

Tabela 6: CEC2014 F4, 10 zmiennych ($f^* = 400$).

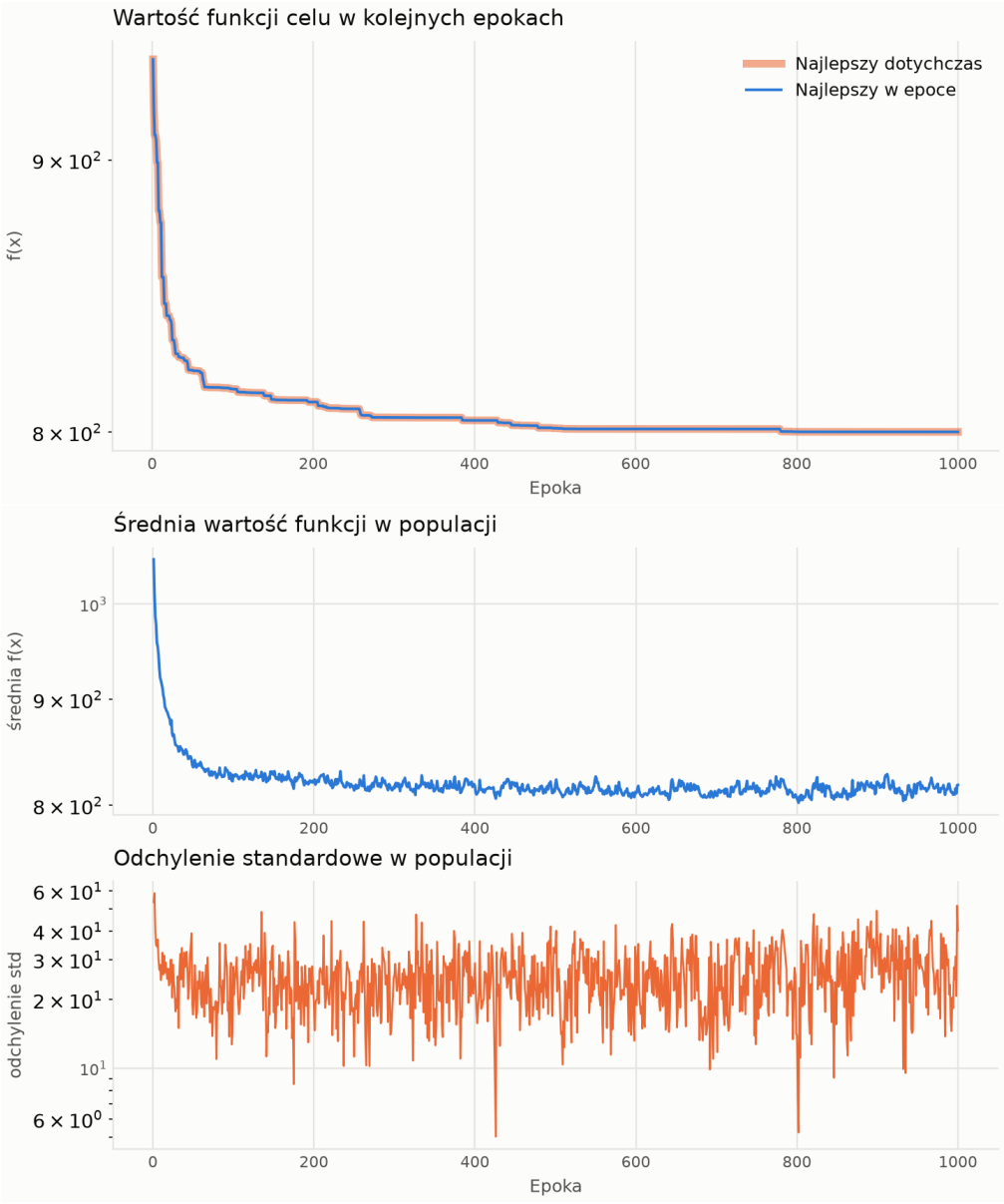


Rysunek 13: CEC2014 F4, najlepsze uruchomienie (konfiguracja „Koło ruletki”, uruchomienie 6 z 10, wynik 401.0450, czas 0.89 s).

8.5 CEC2014 F8, 10 zmiennych

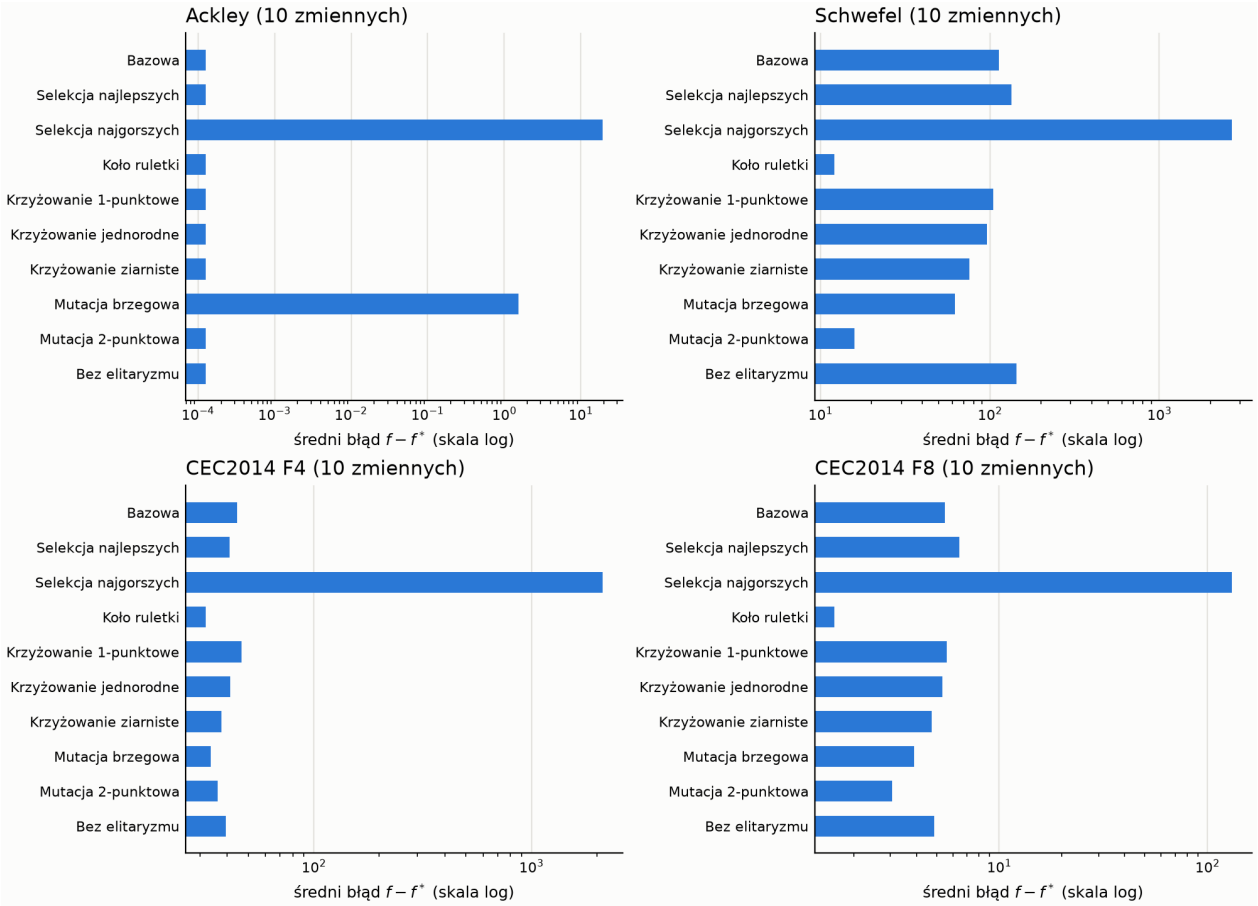
Konfiguracja	Średnia	Najlepszy	Najgorszy	Odch. std	Czas [s]
Bazowa	805.4741	801.6590	810.2613	2.9732	0.77
Selekcja najlepszych	806.4251	803.1767	808.6569	2.1919	0.78
Selekcja najgorszych	931.1137	909.8464	949.5407	10.7731	0.77
Koło ruletki	**801.6200**	800.0616	804.2362	1.1585	0.78
Krzyżowanie 1-punktowe	805.6207	802.2713	809.2861	2.0531	0.64
Krzyżowanie jednorodne	805.3377	803.0810	808.7675	1.8622	0.67
Krzyżowanie ziarniste	804.7352	802.2163	807.4918	1.6873	0.69
Mutacja brzegowa	803.8971	801.1481	806.5758	1.6417	0.82
Mutacja 2-punktowa	803.0686	800.8127	805.1557	1.3429	0.84
Bez elitaryzmu	804.8867	802.5756	809.1788	2.2618	0.80

Tabela 7: CEC2014 F8, 10 zmiennych ($f^* = 800$).



Rysunek 14: CEC2014 F8, najlepsze uruchomienie (konfiguracja „Koło ruletki”, uruchomienie 4 z 10, wynik 800.0616, czas 0.78 s).

8.6 Porównanie konfiguracji



Rysunek 15: Średni błąd $f - f^*$ z 10 uruchomień dla każdej konfiguracji (skala logarytmiczna, mniej znaczy lepiej).

8.7 Porównanie czasu obliczeń

Czasy w tabelach to średni czas jednego uruchomienia (1000 epok, populacja 100). Przy 10 zmiennych jedno uruchomienie trwa od 0.54 do 0.91 s. Najwięcej zależy od metody krzyżowania: krzyżowanie jednopunktowe, jednorodne i ziarniste jest o około 15 do 20% szybsze od dwupunktowego (np. Schwefel: 0.54 s zamiast 0.67 s). Mutacja dwupunktowa jest nieco wolniejsza od jednopunktowej (0.72 s zamiast 0.67 s), a metoda selekcji i wyłączenie elity prawie nie wpływają na czas. Funkcje CEC liczą się dłużej niż Ackley i Schwefel (0.77 do 0.85 s w konfiguracji bazowej zamiast 0.67 do 0.71 s), bo każde wywołanie funkcji wykonuje przesunięcie i obrót punktu.

8.8 Wpływ liczby zmiennych

Konfigurację bazową uruchomiliśmy również dla 20 i 30 zmiennych (po 10 razy).

Funkcja	n	Średnia	Najlepszy	Najgorszy	Czas [s]
Ackley	10	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	0.71
Ackley	20	0.1425	$1.251 \cdot 10^{-4}$	1.4236	0.91
Ackley	30	1.9653	1.5022	2.3261	1.15
Schwefel	10	113.3686	0.3131	347.8354	0.67
Schwefel	20	420.9876	187.5353	683.7104	0.83
Schwefel	30	1041.0689	501.1972	1478.8103	1.00
CEC2014 F4	10	444.4991	419.1238	469.0697	0.85
CEC2014 F4	20	477.0327	426.2379	540.1218	0.90
CEC2014 F4	30	683.0022	584.5078	861.1817	0.91
CEC2014 F8	10	805.4741	801.6590	810.2613	0.77
CEC2014 F8	20	823.4414	815.6099	828.8569	0.80
CEC2014 F8	30	842.9763	832.8916	854.8559	0.82

Tabela 8: Konfiguracja bazowa dla 10, 20 i 30 zmiennych.

9 Wnioski

- Funkcja Ackley jest dla algorytmu łatwa: przy 10 zmiennych wszystkie rozsądne konfiguracje trafiają w optimum z dokładnością ograniczoną tylko przez rozdzielczość kodowania binarnego. Przy 30 zmiennych chromosom ma 600 bitów i 1000 epok to za mało, by zawsze wyjść z minimów lokalnych.
- Selekcja najgorszych wypada najsłabiej na wszystkich funkcjach: rodzicami zostają wyłącznie słabe osobniki, więc postęp zapewnia praktycznie tylko elita. Mutacja brzegowa zmienia tylko pierwszy lub ostatni bit chromosomu, więc przy funkcji Ackley zbyt słabo eksploruje przestrzeń.
- Na funkcjach wielomodalnych (Schwefel, F4, F8) najlepsze średnie daje koło ruletki. Wywiera ono mniejszą presję selekcyjną niż turniej i dłużej utrzymuje różnorodność populacji, co widać na Schwefelu: średni błąd spada z około 113 do około 12.
- Silniejsza mutacja (dwupunktowa) wyraźnie pomaga na funkcji Schwefel i F8, ponieważ ułatwia przeskakiwanie między minimami lokalnymi.
- Rodzaj krzyżowania ma mniejszy wpływ na wynik niż selekcja i mutacja. Krzyżowanie jednopunktowe jest najszybsze, ziarniste daje nieco lepsze średnie na funkcjach wielomodalnych.
- Wyłączenie elitaryzmu wyraźnie pogorszyło wyniki na funkcji Schwefel (średnia 143.7 zamiast 113.4, najlepszy wynik 27.2 zamiast 0.31), bo najlepsze rozwiązanie może zostać utracone. Na funkcjach F4 i F8 różnica mieści się w rozrzucie wynikającym z losowości.
- Wraz ze wzrostem liczby zmiennych błąd szybko rośnie, a czas obliczeń rośnie wolniej (głównie przez dłuższe chromosomy). Dla funkcji CEC przy 20 i 30 zmiennych algorytm nie znajduje optimum przy tej samej liczbie epok i wielkości populacji.
- Funkcje CEC są trudniejsze: na F4 najlepszy wynik to 401.05 (błąd 1.05), na F8 800.06 (błąd 0.06), jednak średnie z 10 uruchomień są wyraźnie gorsze, co pokazuje dużą rolę losowości.

10 Podsumowanie

W tabeli zestawiono najlepszy wynik spośród wszystkich uruchomień dla danej funkcji i liczby zmiennych. Błąd to różnica między osiągniętą a rzeczywistą wartością optimum.

Funkcja	n	Optimum f^*	Wynik AG	Błąd	Konfiguracja
Ackley	10	$4.441 \cdot 10^{-16}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	Bazowa
Ackley	20	$4.441 \cdot 10^{-16}$	$1.251 \cdot 10^{-4}$	$1.251 \cdot 10^{-4}$	Bazowa
Ackley	30	$4.441 \cdot 10^{-16}$	1.5022	1.5022	Bazowa
Schwefel	10	$1.273 \cdot 10^{-4}$	0.0020	0.0019	Koło ruletki
Schwefel	20	$2.546 \cdot 10^{-4}$	187.5353	187.5351	Bazowa
Schwefel	30	$3.818 \cdot 10^{-4}$	501.1972	501.1968	Bazowa
CEC2014 F4	10	400.0000	401.0450	1.0450	Koło ruletki
CEC2014 F4	20	400.0000	426.2379	26.2379	Bazowa
CEC2014 F4	30	400.0000	584.5078	184.5078	Bazowa
CEC2014 F8	10	800.0000	800.0616	0.0616	Koło ruletki
CEC2014 F8	20	800.0000	815.6099	15.6099	Bazowa
CEC2014 F8	30	800.0000	832.8916	32.8916	Bazowa

Tabela 9: Podsumowanie: rzeczywiste optimum, najlepszy wynik algorytmu genetycznego i błąd.

Nagranie przedstawiające działanie aplikacji zostało dołączone do archiwum z projektem.